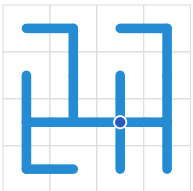
Pipes & Light Up

Search algorithms for two logic puzzles

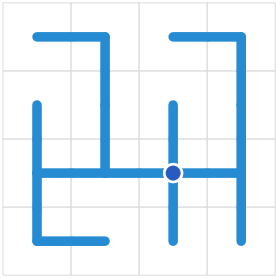


Introduction to Artificial Intelligence — Assignment 1 (HK1 2026–2027)

DFS (blind) + Greedy Best-First (Pipes) · DFS (blind) + A* (Light Up)

[Group — member names go here]

Goal: one connected, acyclic network



- Every port matches its neighbor.
- No port leaves the grid.
- All tiles connect. No cycles.

Tiles are vertices, connections are edges.

$$G \text{ is a tree} \quad \Longrightarrow \quad |E| = n - 1$$

The blue source anchors water visualization.

Rules: puzzle-pipes.com. Goal validation: model.py and search.py.

Problem formulation

Input	$n = m^2$ tiles, fixed shapes, ordinary boundaries
State	Prefix $s = (a_0, \ldots, a_{k-1})$ of assigned orientations
Initial state	Empty prefix (), with all tile shapes known
Action	Assign a rotation to tile k in row-major order
Transition	$s' = s + (a_k)$, accepted after feasibility checks
Goal	$k = n$ and the complete board passes all rules
Cost	Unit assignment cost, unused by either ranking

A solver assignment can represent several manual clicks.
Any valid solution is acceptable. Minimum-click solutions are outside scope.

Implemented state model: Search in pipes_problem/search.py.

Legal successors

The next tile has a fixed shape. Search chooses its orientation.



NE
reject



ES
keep



SW
reject



NW
reject

Top-left corner example. The left and top borders forbid N and W.

Neighbor matching also checks absent ports.
A candidate must pass every shared pruning check before search accepts it.

Example is schematic. Orientation filtering: boundary_domains() and compatible().

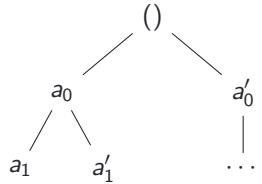
Search space

Distinct rotations per tile

End, elbow, T-junction	4
Straight	2
Cross	1

$$U = \prod_{i=0}^{n-1} |R_i| \leq 4^n$$

Prefix tree



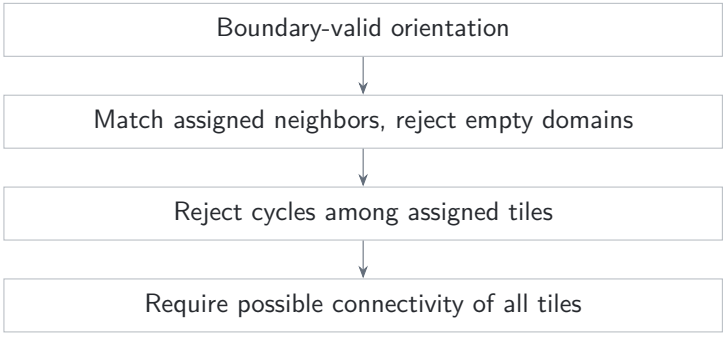
Depth n, branching factor at most 4.
Each prefix has one parent.

Complete assignments before boundary checks.

For the shipped 4×4 example: $U = 268{,}435{,}456$.
This bound counts assignments, not measured expansions.

Count derived from distinct rotations in pipes-4x4-261.json.

Shared pruning



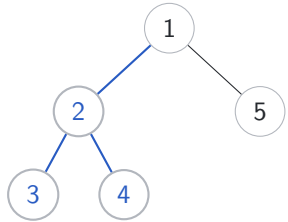
Both solvers use identical checks and fixed tile order.

Root condition: total ports $= 2(n-1)$.
Possible connectivity is necessary but does not prove a solution exists.

Pruning implementation: Search.compatible(). Full reasoning: docs/pipes-analysis.md.

Depth-first search

```
VISIT(s):
  if budget exhausted: return LIMIT
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):
      r = VISIT(child)
      if r is SOLVED or LIMIT: return r
  return UNSOLVABLE
```



Schematic visit order.
Explore a branch, then backtrack.

Finite-tree completeness with sufficient resources and sound pruning.
Worst-case exploration remains exponential.

Simplified pseudocode matches dfs(). Root feasibility precedes VISIT. Cancellation is external.

Greedy's heuristic: remaining choices

$$h(s) = \sum_{i \text{ unassigned}} \log_2 |D_i(s)|$$

$D_i(s)$ contains boundary-valid orientations matching assigned neighbors.

Candidate	Remaining domain sizes	$h(s)$
A	$(1, 2, 4)$	$0 + 1 + 2 = 3$
B	$(1, 1, 2)$	$0 + 0 + 1 = 1$

Greedy prefers B. Smaller values indicate fewer unresolved choices.
Tie order: deeper prefix, then earlier insertion.

The score ignores dependencies between unassigned tiles.
It provides no admissibility or minimum-click guarantee.

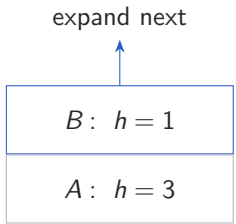
Hypothetical domain example. Exact priority: (h, -depth, insertion order) in greedy().

Greedy Best-First search

```
push () into min-priority queue
while queue is not empty:
  if budget exhausted: return LIMIT
  s = pop minimum-priority state
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    D = FEASIBLE(child)
    if D exists:
      push child with (h(child),
                       -|child|, serial)
return UNSOLVABLE
```

Root feasibility precedes the queue. Priority uses h alone. The unique root starts at zero.

Queue selection



All feasible prefixes remain queued.

Memory can grow exponentially.

Evaluation protocol

- Same nine saved maps for both algorithms.
- Three maps per size, five repeats per algorithm.
- Shared pruning, tile order and resource limits.

Saved map → Timed headless solve → Separate memory replay

Time excludes drawing and playback. Peak memory counts traced Python allocations. Combine passes only when outcomes agree.

Limits: 30 seconds of solver computation and 200,000 expansions.

Runner: benchmark.py. Measurement: measure() and worker.py.

Baseline results

Size	Algorithm	Solved/runs	Time (ms)	Peak (KiB)
4×4	DFS	15/15	3.149	26.984
	Greedy	15/15	3.097	12.359
5×5	DFS	15/15	5.261	53.297
	Greedy	15/15	6.157	19.219
7×7	DFS	15/15	24.469	145.852
	Greedy	15/15	31.695	27.594

Median over 15 successful runs per size and algorithm.
Each group contains three distinct inputs with five timing repeats.

DFS wins at 5×5 and two of three 7×7 maps; Greedy takes the hardest map (38.2 vs 40.8 ms) and holds 4–5× less traced memory.
These inputs are easy under shared pruning; no general winner is claimed.

Measured 25 October 2026, Linux, Python 3.13. Raw rows: docs/benchmarks/2026-10-25-pipes.csv.

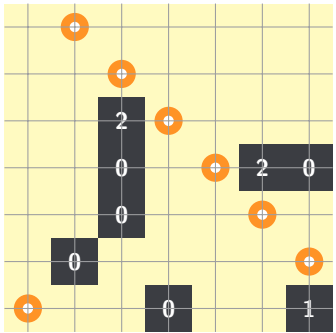
Scope and next experiments

- Compare harder maps and proven-unsolvable inputs.
- Measure the effect of shared pruning separately.
- Keep UNSOLVABLE distinct from LIMIT or CANCELLED.
- Try alternative heuristics (MRV) and broader input sets.

References

Pipes rules: `https://www.puzzle-pipes.com/`
Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Chapters 3 and 6.
`https://aima.cs.berkeley.edu/`

Problem 2: Light Up (Akari)



- Place bulbs on **white cells**; each bulb shines in 4 directions until a wall or the edge.
- Every white cell must be illuminated.
- No two bulbs may see each other.
- A wall numbered k has exactly k adjacent bulbs.

Real sample board: `akari-7-easy-70097` from the generator;
the picture shows its 7-bulb solution.

Rules: puzzle-light-up.com. Board: generator/generate.py, seed 70097.

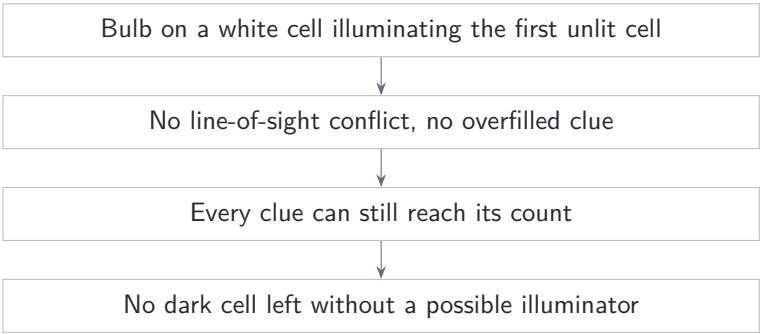
Problem formulation

Input	Board: `None` (white), -1 (wall), 0..4 (numbered wall)
State	Bulb set encoded as a W-**bit bitmask** (W = white cells)
Initial state	Empty set (mask 0)
Action	Place a bulb on an eligible white cell
Transition	Set the bit; reject rule-breaking or unreachable successors
Goal	All white cells lit, all clues exact, no conflicts
Cost	One per placed bulb (A*'s objective)

Both solvers branch on the **first unlit cell (row-major)**: only legal bulbs illuminating it are considered.
One parent per state $\Rightarrow$ no complex duplicate machinery; the comparison between solvers stays fair.

Implemented model: _View.branch_cell in light_up_problem/algorithms/search.py.

Shared sound pruning



The checks only discard states that cannot reach a goal $\Rightarrow$ completeness preserved.
This is why "blind" DFS is fast on small and medium boards.

Implementation: `_View.feasible()` in `algorithms/search.py`.

Blind search: Depth-First Search

```
DFS(puzzle, limits):
  push mask 0 onto the stack
  while stack is not empty:
    if budget exhausted: return LIMIT
    state = stack.pop()
    if state in visited: continue
    if GOAL(state): return SOLVED(state)
    cell = FIRST_UNLIT_CELL(state)
    for c in bulbs illuminating cell:
      child = state | bit(c)
      if FEASIBLE(child): push child
  return UNSOLVABLE
```

LIFO stack + visited set.

Fixed branching order,
no heuristic information.

Worst case $O(2^W)$ states.

Returns the **first** solution — not minimum-bulb.

Simplified pseudocode matches depth_first_search(). TIMEOUT/NODE_LIMIT/MEMORY_LIMIT stay distinct from UNSOLVABLE.

A*'s heuristic: three lower bounds, take the max

$$h(n) = \max\left\{ \left\lceil \frac{D}{G_{\max}} \right\rceil,\ \max_{w} d_w,\ \left\lceil \frac{\sum_w d_w}{H_{\max}} \right\rceil \right\} \qquad f = g + h, \quad g = \#\text{bulbs}$$

D: remaining dark cells; $G_{\max}$: most cells one legal bulb lights;
d_w: numbered wall w's deficit; $H_{\max}$: most adjacencies one bulb satisfies.

Component	Why it lower-bounds
$\lceil D/G_{\max} \rceil$	One bulb lights at most $G_{\max}$ dark cells
$\max_w d_w$	A wall short by d_w needs at least d_w bulbs
$\lceil \sum_w d_w/H_{\max} \rceil$	The total deficit needs that many bulbs

Max of lower bounds is a lower bound $\Rightarrow$ h is **admissible** $\Rightarrow$ A* returns a **minimum-bulb** solution when it completes.

Implementation: _View.lower_bound() in algorithms/search.py.

A* search

```
ASTAR(puzzle, limits):
  push (f=h(0), g=0, mask 0) into min-heap
  best_g = {0: 0}
  while heap is not empty:
    if budget exhausted: return LIMIT
    pop state with minimum f
    if GOAL(state): return SOLVED(state)
    for c in bulbs illuminating
        FIRST_UNLIT_CELL(state):
      child = state | bit(c)
      if FEASIBLE(child) and
         g+1 < best_g.get(child, inf):
        best_g[child] = g+1
        push (g+1+h(child), g+1, child)
  return UNSOLVABLE
```

Simplified pseudocode matches astar(). Unit costs make f monotone along paths.

The min-heap keeps the **whole frontier**.

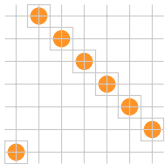
The best_g table fights rediscovery.

Each expansion recomputes three bounds over large bitmasks.

Memory can grow exponentially — visible in the data.

Seeded inputs with a verified witness

- Build a cyclic *witness*: exactly one bulb per row and column.
- Propose walls only where the witness survives; derive clue numbers from adjacent witness bulbs.
- Keep a wall only if the witness *remains a valid solution* (checked with the same `is_goal_state` used in play).



4 sizes: 7, 10, 14, 25

3 difficulties: 16%/10.5%/5.5% walls

12 sample sets covering all combinations

JSON `lightup-puzzle-v1`

Uniqueness not checked

Difficulty labels are structural (wall density), not calibrated against solve times.

Implementation: generator/generate.py. The witness is not stored in the player board.

Visual demo in pygame

- **Manual play**: click to place or remove bulbs; lit cells highlighted, conflicts outlined in red, walls show live counts.
- **Live search**: Run / Pause / Step / Stop with expanded / generated / frontier counters and the current bulb set (sampled events).
- **Compare DFS and A*** on the same board: time, nodes and bulb counts side by side.
- **Play solution**: replay the returned placements step by step.

Solvers run in a separate spawn-safe process so the UI never stalls;
solver time/node/memory limits apply inside the demo too.

Implementation: ui/app.py + worker.py.

Measurement protocol

- 12 sample inputs: 4 sizes $\times$ 3 difficulties, 3 repeats per algorithm.
- Limits: 30 s, 500,000 nodes, 512 MB estimated memory.
- Same boards and limits for DFS and A*.

Seeded board → Solver's own `perf_counter` → `tracemalloc`

Same machine and day as the Pipes tables (Linux, Python 3.13.14) so both puzzles are comparable.
72 raw runs: `docs/benchmarks/2026-10-25-lightup.csv`.

Runner: light_up_problem.benchmark. Evidence: docs/benchmarks/2026-10-25-lightup.md.

Results: DFS is fast and light, A* is optimal but expensive

Input	DFS: time / nodes	A*: time / nodes	Bulbs (DFS → A*)
7 easy	0.026 s / 8	0.273 s / 70	$7 = 7$
7 normal	0.016 s / 9	0.236 s / 153	$8 \rightarrow \mathbf{7}$
7 hard	0.027 s / 7	6.77 s / 3164	$6 = 6$
10 easy	0.549 s / 248	3.50 s / 618	$11 \rightarrow \mathbf{10}$
10 normal	0.057 s / 13	26.7 s / 5464	$12 \rightarrow \mathbf{9}$
10 hard	0.078 s / 12	TIMEOUT 30 s (3621 KiB)	—
14 (all)	0.35–0.42 s / 15–17	TIMEOUT ×3	—
25 (all)	6.7–9.7 s / 33–35	TIMEOUT ×3	—

- DFS solves **12/12** thanks to "first unlit" branching + pruning: almost no backtracking (nodes $\approx 1.3\times$ bulbs).
- A* completes **5/12**, 9–250× slower, but proves optimality: 10-normal needs only 9 bulbs, not 12 (DFS overshoots 33%).
- Rerunning A* with a **600 s** budget: all 7 inputs still TIMEOUT (10-hard: 151k nodes, ∼70 MiB) — TIMEOUT is *inconclusive*, never unsolvable.

Median of 3 runs (30 s) plus one 600 s run each, measured 25 October 2026, Linux. Data: docs/benchmarks/2026-10-25-lightup*.csv.

Limitations and next steps

- Difficulty labels come from wall density, not calibrated solve times.
- Solution uniqueness is not checked.
- tracemalloc measures peak Python allocations, not process RSS.
- Stronger lower bounds (matching over dark cells) could push A* further.

References

Akari rules: `https://www.puzzle-light-up.com/`
Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Chapter 3.
Full analysis: `docs/lightup-analysis.md`.

Summary

- Two puzzles, four algorithms, one shared state-space formulation style.
- Every algorithm ships a step-by-step visual demo and was measured on the same inputs.
- DFS + sound pruning is fast and light on moderate inputs; heuristic search pays bookkeeping costs for better ordering (Greedy) or optimal cost (A*).
- Full data: `docs/benchmarks/2026-10-25-*.csv`.

References

`https://www.puzzle-pipes.com/` · `https://www.puzzle-light-up.com/`
Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed.
Implementation and analysis: `pipes_problem/`, `light_up_problem/`, `docs/`.